

1. Introduction to SQL

LAB EXERCISES:

Lab 3: Create a database called library_db and a table books with columns: book_id, title, author, publisher, year_of_publication, and price. Insert five records into the table.

CREATE DATABASE library_db;

CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(100) NOT NULL, author VARCHAR(50) NOT NULL, publisher VARCHAR(50), year_of_publication YEAR, price DECIMAL(8,2));

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0216 seconds.)

```
CREATE TABLE books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(100) NOT NULL, author VARCHAR(50) NOT NULL, publisher VARCHAR(50), year_of_publication YEAR, price DECIMAL(8,2));
```

INSERT INTO books (title, author, publisher, year_of_publication, price) VALUES

('The Alchemist', 'Paulo Coelho', 'HarperOne', 1988, 250.00)

INSERT INTO books (title, author, publisher, year_of_publication, price) VALUES

('1984', 'George Orwell', 'Secker & Warburg', 1949, 300.00)

INSERT INTO books (title, author, publisher, year_of_publication, price) VALUES

('To Kill a Mockingbird', 'Harper Lee', 'J.B. Lippincott & Co.', 1960, 350.00)

INSERT INTO books (title, author, publisher, year_of_publication, price) VALUES

('The Great Gatsby', 'F. Scott Fitzgerald', 'Scribner', 1925, 280.00)

INSERT INTO books (title, author, publisher, year_of_publication, price) VALUES

('Moby Dick', 'Herman Melville', 'Harper & Brothers', 1851, 400.00);

←T→	book_id	title	author	publisher	year_of_publication	price
☐ Edit ☐ Copy ☐ Delete	1	The Alchemist	Paulo Coelho	HarperOne	1988	250.00
☐ Edit ☐ Copy ☐ Delete	2	1984	George Orwell	Secker & Warburg	1949	300.00
☐ Edit ☐ Copy ☐ Delete	3	To Kill a Mockingbird	Harper Lee	J.B. Lippincott & Co.	1960	350.00
☐ Edit ☐ Copy ☐ Delete	4	The Great Gatsby	F. Scott Fitzgerald	Scribner	1925	280.00
☐ Edit ☐ Copy ☐ Delete	5	Moby Dick	Herman Melville	Harper & Brothers	0000	400.00

CREATE TABLE members (member_id INT PRIMARY KEY AUTO_INCREMENT, member_name VARCHAR(100), date_of_membership DATE, email VARCHAR(100));

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0092 seconds.)

```
CREATE TABLE members (member_id INT PRIMARY KEY AUTO_INCREMENT, member_name VARCHAR(100), date_of_membership DATE, email VARCHAR(100));
```

INSERT INTO members (member_id, member_name, date_of_membership, email) VALUES

(1, 'Rana Vrujal', '2025-01-10', 'rana.vrujal@example.com');

INSERT INTO members (member_id, member_name, date_of_membership, email) VALUES

(2, 'Jaydeep Dama', '2025-02-15', 'jaydeep.dama@example.com');

```
INSERT INTO members (member_id, member_name, date_of_membership, email) VALUES
```

```
(3, 'Meera Patel', '2025-03-05', 'meera.patel@example.com');
```

```
INSERT INTO members (member_id, member_name, date_of_membership, email) VALUES
```

```
(4, 'Kiran Shah', '2025-04-12', 'kiran.shah@example.com');
```

```
INSERT INTO members (member_id, member_name, date_of_membership, email) VALUES
```

```
(5, 'Anjali Desai', '2025-05-20', 'anjali.desai@example.com');
```

		member_id	member_name	date_of_membership	email
☐	Edit Copy Delete	1	Rana Vrujal	2025-01-10	rana.vrujal@example.com
☐	Edit Copy Delete	2	Jaydeep Dama	2025-02-15	jaydeep.dama@example.com
☐	Edit Copy Delete	3	Meera Patel	2021-03-05	meera.patel@example.com
☐	Edit Copy Delete	4	Kiran Shah	2025-04-12	kiran.shah@example.com
☐	Edit Copy Delete	5	Anjali Desai	2020-05-20	anjali.desai@example.com

2. SQL Syntax

LAB EXERCISES:

Lab 3: Retrieve all members who joined the library before 2022. Use appropriate SQL syntax with WHERE and ORDER BY.

		member_id	member_name	date_of_membership	email
☐	Edit Copy Delete	5	Anjali Desai	2020-05-20	anjali.desai@example.com
☐	Edit Copy Delete	3	Meera Patel	2021-03-05	meera.patel@example.com

Lab 4: Write SQL queries to display the titles of books published by a specific author. Sort the results by year_of_publication in descending order.

```
SELECT title FROM books WHERE author = 'Paulo Coelho' ORDER BY year_of_publication DESC;
```

		title
☐	Edit Copy Delete	The Alchemist

3. SQL Constraints

LAB EXERCISES:

Lab 3: Add a CHECK constraint to ensure that the price of books in the books table is greater than 0.

```
ALTER TABLE books ADD CONSTRAINT chk_price_positive CHECK (price > 0);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0408 seconds.)

```
ALTER TABLE books ADD CONSTRAINT chk_price_positive CHECK (price > 0);
```

Lab 4: Modify the members table to add a UNIQUE constraint on the email column, ensuring that each member has a unique email address.

```
ALTER TABLE members ADD CONSTRAINT unique_email UNIQUE (email);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0205 seconds.)

```
ALTER TABLE members ADD CONSTRAINT unique_email UNIQUE (email);
```

4. Main SQL Commands and Sub-commands (DDL)

LAB EXERCISES:

Lab 3: Create a table authors with the following columns: author_id, first_name, last_name, and country. Set author_id as the primary key.

```
CREATE TABLE authors (author_id INT PRIMARY KEY AUTO_INCREMENT, first_name VARCHAR(50), last_name VARCHAR(50), country VARCHAR(50));
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(1, 'Paulo', 'Coelho', 'Brazil');
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(2, 'George', 'Orwell', 'United Kingdom');
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(3, 'Harper', 'Lee', 'United States');
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(4, 'F. Scott', 'Fitzgerald', 'United States');
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(5, 'Herman', 'Melville', 'United States');
```

```
SELECT * FROM authors;
```

	author_id	first_name	last_name	country
☐ Edit Copy Delete	1	Paulo	Coelho	Brazil
☐ Edit Copy Delete	2	George	Orwell	United Kingdom
☐ Edit Copy Delete	3	Harper	Lee	United States
☐ Edit Copy Delete	4	F. Scott	Fitzgerald	United States
☐ Edit Copy Delete	5	Herman	Melville	United States

Lab 4: Create a table publishers with columns: publisher_id, publisher_name, contact_number, and address. Set publisher_id as the primary key and contact_number as unique.

```
CREATE TABLE publishers ( publisher_id INT PRIMARY KEY,publisher_name VARCHAR(100),contact_number VARCHAR(20) UNIQUE,address VARCHAR(200));
```

```
INSERT INTO publishers (publisher_id, publisher_name, contact_number, address) VALUES
```

```
(1, 'HarperOne', '1234567890', '123 Main St, New York');
```

```
INSERT INTO publishers (publisher_id, publisher_name, contact_number, address) VALUES
```

```
(2, 'Secker & Warburg', '2345678901', '45 High St, London');
```

```
INSERT INTO publishers (publisher_id, publisher_name, contact_number, address) VALUES
```

```
(3, 'J.B. Lippincott & Co.', '3456789012', '67 Market St, Philadelphia');
```

```
INSERT INTO publishers (publisher_id, publisher_name, contact_number, address) VALUES
```

```
(4, 'Scribner', '4567890123', '89 Broadway, New York');
```

```
INSERT INTO publishers (publisher_id, publisher_name, contact_number, address) VALUES
```

```
(5, 'Harper & Brothers', '5678901234', '101 Wall St, New York');
```

```
SELECT * FROM publishers;
```

← T →				publisher_id	publisher_name	contact_number	address
☐	 Edit	 Copy	 Delete	1	HarperOne	1234567890	123 Main St, New York
☐	 Edit	 Copy	 Delete	2	Secker & Warburg	2345678901	45 High St, London
☐	 Edit	 Copy	 Delete	3	J.B. Lippincott & Co.	3456789012	67 Market St, Philadelphia
☐	 Edit	 Copy	 Delete	4	Scribner	4567890123	89 Broadway, New York
☐	 Edit	 Copy	 Delete	5	Harper & Brothers	5678901234	101 Wall St, New York

5. ALTER Command

LAB EXERCISES:

Lab 3: Add a new column genre to the books table. Update the genre for all existing records.

```
ALTER TABLE books ADD genre VARCHAR(50);
```

```
UPDATE books SET genre = 'Fiction' WHERE book_id = 1;
```

```
UPDATE books SET genre = 'Dystopian' WHERE book_id = 2;
```

```
UPDATE books SET genre = 'Classic' WHERE book_id = 3;
```

```
UPDATE books SET genre = 'Classic' WHERE book_id = 4;
```

```
UPDATE books SET genre = 'Adventure' WHERE book_id = 5;
```

	book_id	title	author	publisher	year_of_publication	price	genre
☐ Edit Copy Delete	1	The Alchemist	Paulo Coelho	HarperOne	1988	250.00	Fiction
☐ Edit Copy Delete	2	1984	George Orwell	Secker & Warburg	1949	300.00	Dystopian
☐ Edit Copy Delete	3	To Kill a Mockingbird	Harper Lee	J.B. Lippincott & Co.	1960	350.00	Classic
☐ Edit Copy Delete	4	The Great Gatsby	F. Scott Fitzgerald	Scribner	1925	280.00	Classic
☐ Edit Copy Delete	5	Moby Dick	Herman Melville	Harper & Brothers	2000	400.00	Adventure
☐ Edit Copy Delete	7	Invalid Book	Author Y	Publisher Z	2025	100.00	NULL

Lab 4: Modify the members table to increase the length of the email column to 100 characters.

ALTER TABLE members MODIFY email VARCHAR(100);

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0144 seconds.)

ALTER TABLE members MODIFY email VARCHAR(100);

Field	Type	Null	Key	Default	Extra
member_id	int	NO	PRI	NULL	auto_increment
member_name	varchar(100)	YES		NULL	
date_of_membership	date	YES		NULL	
email	varchar(100)	YES	UNI	NULL	

6. DROP Command

LAB EXERCISES:

Lab 3: Drop the publishers table from the database after verifying its structure.

DROP TABLE publishers;

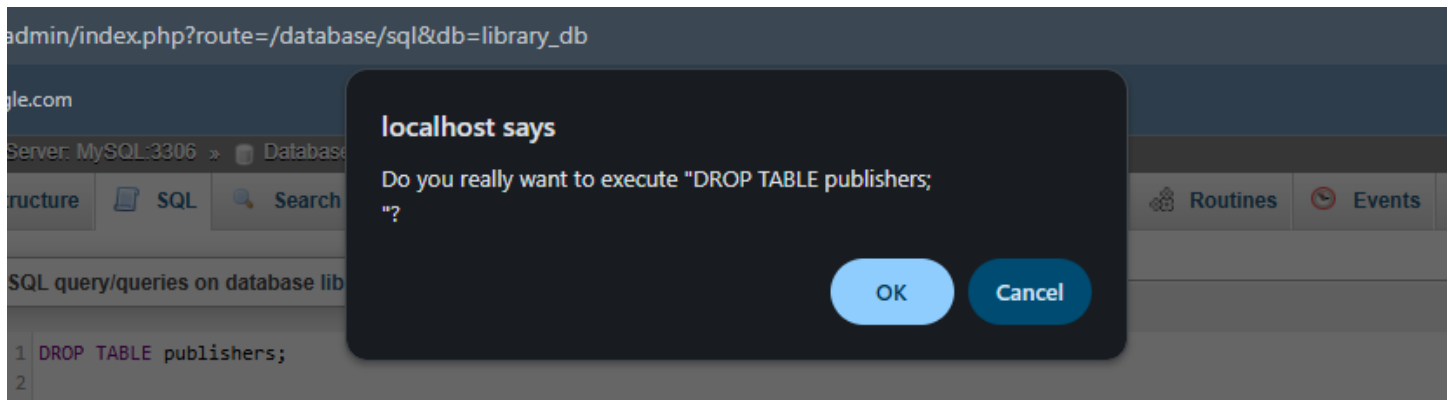


Table	Action	Rows
☐ authors	★ Browse Structure Search Insert Empty Drop	
☐ books	★ Browse Structure Search Insert Empty Drop	
☐ members	★ Browse Structure Search Insert Empty Drop	

```
CREATE TABLE members_backup AS
```

```
SELECT * FROM members;
```

```
SHOW TABLES;
```

Tables_in_library_db

authors

books

members_backup

```
SELECT * FROM `members_backup`
```

member_id	member_name	date_of_membership	email
1	Rana Vrujal	2025-01-10	rana.vrujal@example.com
2	Jaydeep Dama	2025-02-15	jaydeep.dama@example.com
3	Meera Patel	2021-03-05	meera.patel@example.com
4	Kiran Shah	2025-04-12	kiran.shah@example.com
5	Anjali Desai	2020-05-20	anjali.desai@example.com

7. Data Manipulation Language (DML)

LAB EXERCISES:

Lab 4: Insert three new authors into the authors table, then update the last name of one of the authors.

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(6, 'Ravindra', 'Patel', 'India');
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(7, 'Anita', 'Shah', 'India');
```

```
INSERT INTO authors (author_id, first_name, last_name, country) VALUES
```

```
(8, 'Michael', 'Brown', 'USA');
```

author_id	first_name	last_name	country
1	Paulo	Coelho	Brazil
2	George	Orwell	United Kingdom
3	Harper	Lee	United States
4	F. Scott	Fitzgerald	United States
5	Herman	Melville	United States
6	Ravindra	Patel	India
7	Anita	Shah	India
8	Michael	Brown	USA

```
UPDATE authors SET last_name = 'Mehta' WHERE author_id = 7;
```

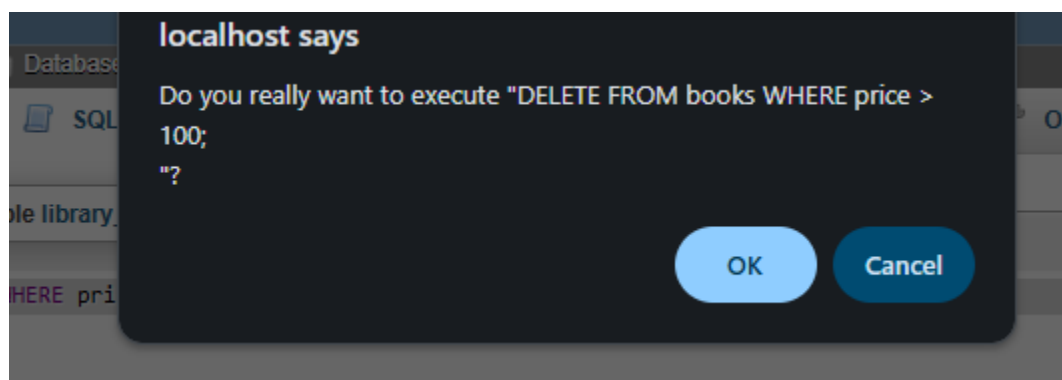
author_id	first_name	last_name	country
1	Paulo	Coelho	Brazil
2	George	Orwell	United Kingdom
3	Harper	Lee	United States
4	F. Scott	Fitzgerald	United States
5	Herman	Melville	United States
6	Ravindra	Patel	India
7	Anita	Mehta	India
8	Michael	Brown	USA

```
SELECT * FROM authors WHERE author_id = 7;
```

author_id	first_name	last_name	country
7	Anita	Mehta	India

Lab 5: Delete a book from the books table where the price is higher than \$100.

```
DELETE FROM books WHERE price > 100;
```



book_id	title	author	publisher	year_of_publication	price	genre
7	Invalid Book	Author Y	Publisher Z	2025	100.00	NULL

8. UPDATE Command

LAB EXERCISES:

Lab 3: Update the year_of_publication of a book with a specific book_id.

```
UPDATE books SET year_of_publication = 2021 WHERE book_id = 7;
```

```
SELECT * FROM books WHERE book_id = 7;
```

book_id	title	author	publisher	year_of_publication	price
7	Invalid Book	Author Y	Publisher Z	2021	100.00

Lab 4: Increase the price of all books published before 2015 by 10%.

```
UPDATE books SET price = price * 1.10 WHERE year_of_publication < 2015;
```

```
SELECT * FROM books WHERE year_of_publication < 2015;
```

book_id	title	author	publisher	year_of_publication	price	genre
10	The Alchemist	Paulo Coelho	HarperCollins	1988	330.00	Fiction
8	The Power of Habit	Charles Duhigg	Random House	2012	495.00	Self-Help

9. DELETE Command

LAB EXERCISES:

Lab 3: Remove all members who joined before 2020 from the members table.

```
SELECT * FROM members WHERE date_of_membership < '2020-01-01';
```

member_id	member_name	date_of_membership	email
2	Jaydeep Dama	2019-11-20	jaydeep@gmail.com
5	Karan Soni	2018-12-05	karan@example.com

Lab 4: Delete all books that have a NULL value in the author column.

```
DELETE FROM books WHERE author IS NULL;
```

```
SELECT * FROM books WHERE author IS NULL;
```

<pre>SELECT * FROM books WHERE author IS NULL;</pre>						
☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]						
book_id	title	author	publisher	year_of_publication	price	genre

10. Data Query Language (DQL)

LAB EXERCISES:

Lab 4: Write a query to retrieve all books with price between \$50 and \$100.

```
SELECT * FROM books WHERE price BETWEEN 50 AND 100;
```

book_id	title	author	publisher	year_of_publication	price	genre
7	Invalid Book	Author Y	Publisher Z	2021	100.00	NULL

Lab 5: Retrieve the list of books sorted by author in ascending order and limit the results to the top 3 entries.

```
SELECT * FROM books ORDER BY author ASC LIMIT 3;
```

book_id	title	author	publisher	year_of_publication	price	genre
7	Invalid Book	Author Y	Publisher Z	2021	100.00	NULL
8	The Power of Habit	Charles Duhigg	Random House	2012	495.00	Self-Help
9	Atomic Habits	James Clear	Penguin	2018	550.00	Self-Help

11. Data Control Language (DCL)

LAB EXERCISES:

Lab 3: Grant SELECT permission to a user named librarian on the books table.

Lab 4: Grant INSERT and UPDATE permissions to the user admin on the members table.

12. REVOKE Command

LAB EXERCISES:

Lab 3: Revoke the INSERT privilege from the user librarian on the books table.

Lab 4: Revoke all permissions from user admin on the members table.

13. Transaction Control Language (TCL)

LAB EXERCISES:

Lab 3: Use COMMIT after inserting multiple records into the books table, then make another insertion and perform a ROLLBACK.

START TRANSACTION;

INSERT INTO books (book_id, title, author, publisher, year_of_publication, price)

VALUES

(11, 'The Power of Habit', 'Charles Duhigg', 'Random House', 2012, 450);

INSERT INTO books (book_id, title, author, publisher, year_of_publication, price)

VALUES(12, 'Atomic Habits', 'James Clear', 'Penguin', 2018, 550);

COMMIT;

			book_id	title	author	publisher	year_of_publication	price	genre
☐				12	Atomic Habits	James Clear	Penguin	2018	550.00 NULL
☐				11	The Power of Habit	Charles Duhigg	Random House	2012	450.00 NULL
☐				7	Invalid Book	Author Y	Publisher Z	2021	100.00 NULL
☐				10	The Alchemist	Paulo Coelho	HarperCollins	1988	330.00 Fiction
☐				9	Atomic Habits	James Clear	Penguin	2018	550.00 Self-Help
☐				8	The Power of Habit	Charles Duhigg	Random House	2012	495.00 Self-Help

START TRANSACTION;

INSERT INTO books (book_id, title, author, publisher, year_of_publication, price)

VALUES

```
(13, 'Educated', 'Tara Westover', 'Random House', 2018, 500);
```

```
ROLLBACK;
```

Lab 4: Set a SAVEPOINT before making updates to the members table, perform some updates, and then roll back to the SAVEPOINT.

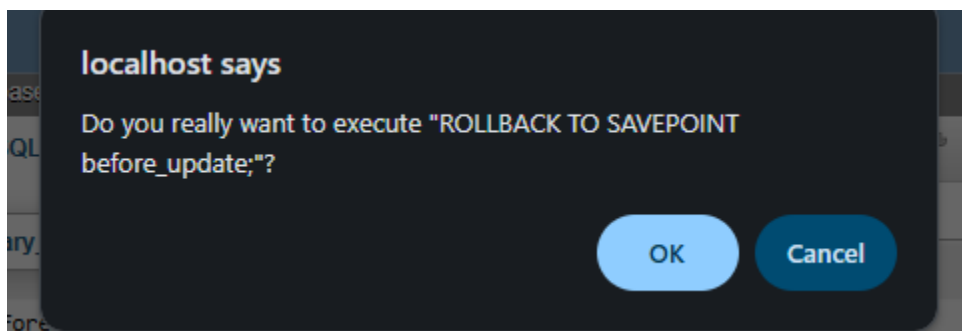
```
START TRANSACTION;
```

```
SAVEPOINT before_update;
```

```
UPDATE members SET member_name = 'Rana V' WHERE member_id = 1;
```

```
UPDATE members SET email = 'jaydeep.dama@example.com' WHERE member_id = 2;
```

```
ROLLBACK TO SAVEPOINT before_update;
```



```
COMMIT;
```

14. SQL Joins

LAB EXERCISES:

Lab 3: Perform an INNER JOIN between books and authors tables to display the title of books and their respective authors' names.

```
SELECT b.title, a.first_name, a.last_name
```

```
FROM books b
```

```
INNER JOIN authors a
```

```
ON b.author = CONCAT(a.first_name, ' ', a.last_name);
```

title	first_name	last_name
The Alchemist	Paulo	Coelho

Lab 4: Use a FULL OUTER JOIN to retrieve all records from the books and authors tables, including those with no matching entries in the other table.

```
SELECT b.title, a.first_name, a.last_name
```

```
FROM books b
```

```
LEFT JOIN authors a
```

```
ON b.author = CONCAT(a.first_name, ' ', a.last_name)
```

UNION

```
SELECT b.title, a.first_name, a.last_name
FROM books b
RIGHT JOIN authors a
ON b.author = CONCAT(a.first_name, ' ', a.last_name);
```

title	first_name	last_name
Atomic Habits	NULL	NULL
The Power of Habit	NULL	NULL
Invalid Book	NULL	NULL
Educated	NULL	NULL
The Alchemist	Paulo	Coelho
NULL	George	Orwell
NULL	Harper	Lee
NULL	F. Scott	Fitzgerald
NULL	Herman	Melville
NULL	Ravindra	Patel
NULL	Anita	Mehta
NULL	Michael	Brown

15. SQL Group By

LAB EXERCISES:

Lab 3: Group books by genre and display the total number of books in each genre.

```
SELECT genre, COUNT(*) AS total_books FROM books GROUP BY genre;
```

genre	total_books
NULL	4
Fiction	1
Self-Help	2

Lab 4: Group members by the year they joined and find the number of members who joined each year.

```
SELECT YEAR(date_of_membership) AS join_year, COUNT(*) AS total_members
FROM members
GROUP BY YEAR(date_of_membership)
ORDER BY join_year;
```

join_year	total_members
2018	1
2019	1
2020	1
2025	2

16. SQL Stored Procedure

LAB EXERCISES:

Lab 3: Write a stored procedure to retrieve all books by a particular author.

DELIMITER \$\$

```
CREATE PROCEDURE get_books_by_author(IN author_name VARCHAR(100))
```

```
BEGIN
```

```
    SELECT *
```

```
    FROM books
```

```
    WHERE author = author_name;
```

```
END $$
```

DELIMITER ;

```
CALL get_books_by_author('Paulo Coelho');
```

book_id	title	author	publisher	year_of_publication	price	genre
10	The Alchemist	Paulo Coelho	HarperCollins	1988	330.00	Fiction

Lab 4: Write a stored procedure that takes book_id as an argument and returns the price of the book.

DELIMITER \$\$

```
CREATE PROCEDURE get_book_price(IN b_id INT, OUT book_price DECIMAL(10,2))
```

```
BEGIN
```

```
    SELECT price INTO book_price
```

```
    FROM books
```

```
    WHERE book_id = b_id;
```

```
END $$
```

DELIMITER ;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0235 seconds.)

```
CREATE PROCEDURE get_book_price(IN b_id INT, OUT book_price DECIMAL(10,2)) BEGIN SELECT price INTO book_price FROM books WHERE book_id = b_id; END;
```

```
CALL get_book_price(9, @price);
```

```
SELECT @price AS book_price;
```

book_price
550.00

17. SQL View

LAB EXERCISES:

Lab 3: Create a view to show only the title, author, and price of books from the books table.

```
CREATE VIEW book_summary AS
```

```
SELECT title, author, price
```

```
FROM books;
```

```
SELECT * FROM book_summary;
```

		title	author	price
☐	Edit Copy Delete	Atomic Habits	James Clear	550.00
☐	Edit Copy Delete	The Power of Habit	Charles Duhigg	450.00
☐	Edit Copy Delete	Invalid Book	Author Y	100.00
☐	Edit Copy Delete	Educated	Tara Westover	500.00
☐	Edit Copy Delete	The Alchemist	Paulo Coelho	330.00
☐	Edit Copy Delete	Atomic Habits	James Clear	550.00
☐	Edit Copy Delete	The Power of Habit	Charles Duhigg	495.00

Lab 4: Create a view to display members who joined before 2020.

```
CREATE VIEW members_before_2020 AS
```

```
SELECT *
```

```
FROM members
```

```
WHERE date_of_membership < '2020-01-01';
```

```
SELECT * FROM members_before_2020;
```

		member_id	member_name	date_of_membership	email
☐	Edit Copy Delete	2	Jaydeep Dama	2019-11-20	jaydeep.dama@example.com
☐	Edit Copy Delete	5	Karan Soni	2018-12-05	karan@example.com

18. SQL Trigger

LAB EXERCISES:

Lab 3: Create a trigger to automatically update the last_modified timestamp of the books table whenever a record is updated.

ALTER TABLE books

ADD COLUMN last_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;

DELIMITER \$\$

CREATE TRIGGER update_last_modified

BEFORE UPDATE ON books

FOR EACH ROW

BEGIN

SET NEW.last_modified = CURRENT_TIMESTAMP;

END \$\$

DELIMITER ;

UPDATE books

SET price = 500

WHERE book_id = 8;

SELECT book_id, title, price, last_modified

FROM books

WHERE book_id = 8;

	book_id	title	price	last_modified
☐ Edit ☐ Copy ☐ Delete	8	The Power of Habit	500.00	2025-12-04 18:34:24

Lab 4: Create a trigger that inserts a log entry into a log_changes table whenever a DELETE operation is performed on the books table.

CREATE TABLE log_changes (

log_id INT AUTO_INCREMENT PRIMARY KEY,

book_id INT,

title VARCHAR(255),

```

operation VARCHAR(50),
change_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
DELIMITER $$
```

```

CREATE TRIGGER log_book_deletion
AFTER DELETE ON books
FOR EACH ROW
BEGIN
    INSERT INTO log_changes (book_id, title, operation)
    VALUES (OLD.book_id, OLD.title, 'DELETE');
END $$

```

```
DELIMITER ;
```

```
DELETE FROM books WHERE book_id = 10;
```

	log_id	book_id	title	operation	change_time
☐ Edit Copy Delete	1	10	The Alchemist	DELETE	2025-12-04 18:37:25

19. Introduction to PL/SQL

LAB EXERCISES:

Lab 3: Write a PL/SQL block to insert a new book into the books table and display a confirmation message.

```
DELIMITER $$
```

```

CREATE PROCEDURE insert_new_book(
    IN p_title VARCHAR(100),
    IN p_author VARCHAR(50),
    IN p_publisher VARCHAR(50),
    IN p_year YEAR,
    IN p_price DECIMAL(8,2)
)

```

BEGIN

INSERT INTO books (title, author, publisher, year_of_publication, price)

VALUES (p_title, p_author, p_publisher, p_year, p_price);

SELECT CONCAT('Book inserted successfully: ', p_title) AS message;

END \$\$

DELIMITER ;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0151 seconds.)

```
CREATE PROCEDURE insert_new_book( IN p_title VARCHAR(100), IN p_author VARCHAR(50), IN p_publisher VARCHAR(50), IN p_year YEAR, IN p_price DECIMAL(8,2) ) BEGIN INSERT INTO
books (title, author, publisher, year_of_publication, price) VALUES (p_title, p_author, p_publisher, p_year, p_price); SELECT CONCAT('Book inserted successfully: ',
p_title) AS message; END;
```

CALL insert_new_book(

'Harry Potter',

'J.K. Rowling',

'Bloomsbury',

1997,

450.00

);

message

Book inserted successfully: Harry Potter

Lab 4: Write a PL/SQL block to display the total number of books in the books table.

DELIMITER \$\$

CREATE PROCEDURE get_total_books()

BEGIN

SELECT COUNT(*) AS total_books FROM books;

END \$\$

DELIMITER ;

CALL get_total_books();

total_books

7

20. PL/SQL Syntax

LAB EXERCISES:

Lab 3: Write a PL/SQL block to declare variables for book_id and price, assign values, and display the results.

DELIMITER \$\$

```
CREATE PROCEDURE display_values()
BEGIN
    DECLARE v_book_id INT DEFAULT 101;
    DECLARE v_price DECIMAL(8,2) DEFAULT 299.50;

    SELECT v_book_id AS book_id, v_price AS price;
END $$
```

DELIMITER ;

CALL display_values();

book_id	price
101	299.50

Lab 4: Write a PL/SQL block using constants and perform arithmetic operations on book prices.

DELIMITER \$\$

```
CREATE PROCEDURE lab4_book_price_operations()
BEGIN
    DECLARE price1 DECIMAL(8,2) DEFAULT 250.00;
    DECLARE price2 DECIMAL(8,2) DEFAULT 150.00;

    DECLARE total_price DECIMAL(8,2);
    DECLARE difference DECIMAL(8,2);
    DECLARE discounted_price DECIMAL(8,2);

    SET total_price = price1 + price2;
    SET difference = price1 - price2;
    SET discounted_price = price1 * 0.90;
```

```

SELECT price1 AS Price_1,
       price2 AS Price_2,
       total_price AS Total_Price,
       difference AS Difference,
       discounted_price AS Discounted_Price;

END $$

```

```

DELIMITER ;

```

```

CALL lab4_book_price_operations();

```

Price_1	Price_2	Total_Price	Difference	Discounted_Price
250.00	150.00	400.00	100.00	225.00

21. PL/SQL Control Structures

LAB EXERCISES:

Lab 3: Write a PL/SQL block using IF-THEN-ELSE to check if a book's price is above \$100 and print a message accordingly.

```

DELIMITER $$

```

```

CREATE PROCEDURE check_book_price()

```

```

BEGIN

```

```

    DECLARE v_price DECIMAL(8,2) DEFAULT 120;

```

```

    IF v_price > 100 THEN

```

```

        SELECT 'The price is above $100' AS message;

```

```

    ELSE

```

```

        SELECT 'The price is $100 or below' AS message;

```

```

    END IF;

```

```

END $$

```

```

DELIMITER ;

```

```

CALL check_book_price();

```

message

The price is above \$100

Lab 4: Use a FOR LOOP in PL/SQL to display the details of all books one by one.

DELIMITER \$\$

```
CREATE PROCEDURE display_all_books()
BEGIN
    DECLARE done INT DEFAULT 0;
    DECLARE v_id INT;
    DECLARE v_title VARCHAR(100);
    DECLARE v_author VARCHAR(100);

    DECLARE book_cursor CURSOR FOR
        SELECT book_id, title, author FROM books;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN book_cursor;

    read_loop: LOOP
        FETCH book_cursor INTO v_id, v_title, v_author;
        IF done = 1 THEN
            LEAVE read_loop;
        END IF;

        SELECT v_id AS Book_ID, v_title AS Title, v_author AS Author;
    END LOOP;

    CLOSE book_cursor;
END $$

DELIMITER ;
```

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0261 seconds.)

```
CREATE PROCEDURE display_all_books() BEGIN DECLARE done INT DEFAULT 0; DECLARE v_id INT; DECLARE v_title VARCHAR(100); DECLARE v_author VARCHAR(100); DECLARE book_cursor CURSOR FOR SELECT book_id, title, author FROM books; DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1; OPEN book_cursor; read_loop: LOOP FETCH book_cursor INTO v_id, v_title, v_author; IF done = 1 THEN LEAVE read_loop; END IF; SELECT v_id AS Book_ID, v_title AS Title, v_author AS Author; END LOOP; CLOSE book_cursor; END;
```

22. SQL Cursors

LAB EXERCISES:

Lab 3: Write a PL/SQL block using an explicit cursor to fetch and display all records from the members table.

DELIMITER \$\$

```
CREATE PROCEDURE display_members()
```

```
BEGIN
```

```
    DECLARE done INT DEFAULT 0;
```

```
    DECLARE v_id INT;
```

```
    DECLARE v_name VARCHAR(100);
```

```
    DECLARE v_join_date DATE;
```

```
    DECLARE member_cursor CURSOR FOR
```

```
        SELECT member_id, member_name, join_date FROM members;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
    OPEN member_cursor;
```

```
    read_loop: LOOP
```

```
        FETCH member_cursor INTO v_id, v_name, v_join_date;
```

```
        IF done = 1 THEN
```

```
            LEAVE read_loop;
```

```
        END IF;
```

```
        SELECT v_id AS Member_ID, v_name AS Member_Name, v_join_date AS Join_Date;
```

```
    END LOOP;
```

```
CLOSE member_cursor;
```

```
END $$
```

```
DELIMITER ;
```

✔ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0241 seconds.)

```
CREATE PROCEDURE display_members() BEGIN DECLARE done INT DEFAULT 0; DECLARE v_id INT; DECLARE v_name VARCHAR(100); DECLARE v_join_date DATE; DECLARE member_cursor CURSOR
FOR SELECT member_id, member_name, join_date FROM members; DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1; OPEN member_cursor; read_loop: LOOP FETCH member_cursor
INTO v_id, v_name, v_join_date; IF done = 1 THEN LEAVE read_loop; END IF; SELECT v_id AS Member_ID, v_name AS Member_Name, v_join_date AS Join_Date; END LOOP; CLOSE
member_cursor; END;
```

Lab 4: Create a cursor to retrieve books by a particular author and display their titles.

```
DELIMITER $$
```

```
DROP PROCEDURE IF EXISTS get_books_by_author$$
```

```
CREATE PROCEDURE get_books_by_author(IN p_author VARCHAR(100))
```

```
BEGIN
```

```
    DECLARE done INT DEFAULT 0;
```

```
    DECLARE v_title VARCHAR(200);
```

```
    DECLARE book_cursor CURSOR FOR
```

```
        SELECT title FROM books WHERE author = p_author;
```

```
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
```

```
    OPEN book_cursor;
```

```
    read_loop: LOOP
```

```
        FETCH book_cursor INTO v_title;
```

```
        IF done = 1 THEN
```

```
            LEAVE read_loop;
```

```
        END IF;
```

```
        SELECT v_title AS Title;
```

```
    END LOOP;
```

```
CLOSE book_cursor;

END$$

DELIMITER ;

CALL get_books_by_author('Author Name');
```

book_id	title	author	publisher	year_of_publication	price	genre	last_modified
13	Atomic Habits	James Clear	Penguin	2018	550.00	NULL	2025-12-04 18:33:03
11	The Power of Habit	Charles Duhigg	Random House	2012	450.00	NULL	2025-12-04 18:33:03
		Author Y	Publisher Z	2021	100.00	NULL	2025-12-04 18:33:03
		Tara Westover	Random House	2018	500.00	NULL	2025-12-04 18:33:03
14	Harry Potter	J.K. Rowling	Bloomsbury	1997	450.00	NULL	2025-12-04 19:05:33
9	Atomic Habits	James Clear	Penguin	2018	550.00	Self-Help	2025-12-04 18:33:03
8	The Power of Habit	Charles Duhigg	Random House	2012	500.00	Self-Help	2025-12-04 18:34:24

23. Rollback and Commit Savepoint

LAB EXERCISES:

Lab 3: Perform a transaction that includes inserting a new member, setting a SAVEPOINT, and rolling back to the savepoint after making updates.

```
START TRANSACTION;

INSERT INTO members (member_name, date_of_membership)
VALUES ('John Patel', '2024-01-10');

SAVEPOINT sp1;

UPDATE members
SET member_name = 'Temp Name'
WHERE member_name = 'John Patel';

ROLLBACK TO sp1;
```

COMMIT;

SELECT * FROM members;

← →		member_id	member_name	date_of_membership	email	join_date
☐	Edit Copy Delete	1	Rana V	2025-05-10	vrujal@gmail.com	2021-01-01
☐	Edit Copy Delete	2	Jaydeep Dama	2019-11-20	jaydeep.dama@example.com	2021-01-01
☐	Edit Copy Delete	3	Amit Patel	2025-02-15	amit@example.com	2021-01-01
☐	Edit Copy Delete	4	Neha Shah	2020-09-30	neha@example.com	2021-01-01
☐	Edit Copy Delete	5	Karan Soni	2018-12-05	karan@example.com	2021-01-01
☐	Edit Copy Delete	0	Temp Name	2024-01-10	NULL	NULL

Lab 4: Use COMMIT after successfully inserting multiple books into the books table, then use ROLLBACK to undo a set of changes made after a savepoint.

START TRANSACTION;

INSERT INTO books (title, author, publisher, year_of_publication, price)

VALUES ('Book A', 'Author X', 'Publisher 1', 2020, 300),

('Book B', 'Author Y', 'Publisher 2', 2021, 450),

('Book C', 'Author Z', 'Publisher 3', 2019, 250);

COMMIT;

START TRANSACTION;

SAVEPOINT sp1;

UPDATE books

SET price = price + 50

WHERE author = 'Author X';

UPDATE books

SET price = price + 100

WHERE author = 'Author Y';

ROLLBACK TO sp1;

COMMIT;

SELECT * FROM books;

		book_id	title	author	publisher	year_of_publication	price	genre	last_modified
☐	 Edit  Copy  Delete	12	Atomic Habits	James Clear	Penguin	2018	550.00	NULL	2025-12-04 18:33:03
☐	 Edit  Copy  Delete	11	The Power of Habit	Charles Duhigg	Random House	2012	450.00	NULL	2025-12-04 18:33:03
☐	 Edit  Copy  Delete	7	Invalid Book	Author Y	Publisher Z	2021	200.00	NULL	2025-12-05 20:43:12
☐	 Edit  Copy  Delete	13	Educated	Tara Westover	Random House	2018	500.00	NULL	2025-12-04 18:33:03
☐	 Edit  Copy  Delete	14	Harry Potter	J.K. Rowling	Bloomsbury	1997	450.00	NULL	2025-12-04 19:05:33
☐	 Edit  Copy  Delete	9	Atomic Habits	James Clear	Penguin	2018	550.00	Self-Help	2025-12-04 18:33:03
☐	 Edit  Copy  Delete	8	The Power of Habit	Charles Duhigg	Random House	2012	500.00	Self-Help	2025-12-04 18:34:24
☐	 Edit  Copy  Delete	15	Book A	Author X	Publisher 1	2020	350.00	NULL	2025-12-05 20:43:12
☐	 Edit  Copy  Delete	16	Book B	Author Y	Publisher 2	2021	550.00	NULL	2025-12-05 20:43:12
☐	 Edit  Copy  Delete	17	Book C	Author Z	Publisher 3	2019	250.00	NULL	2025-12-05 20:43:12